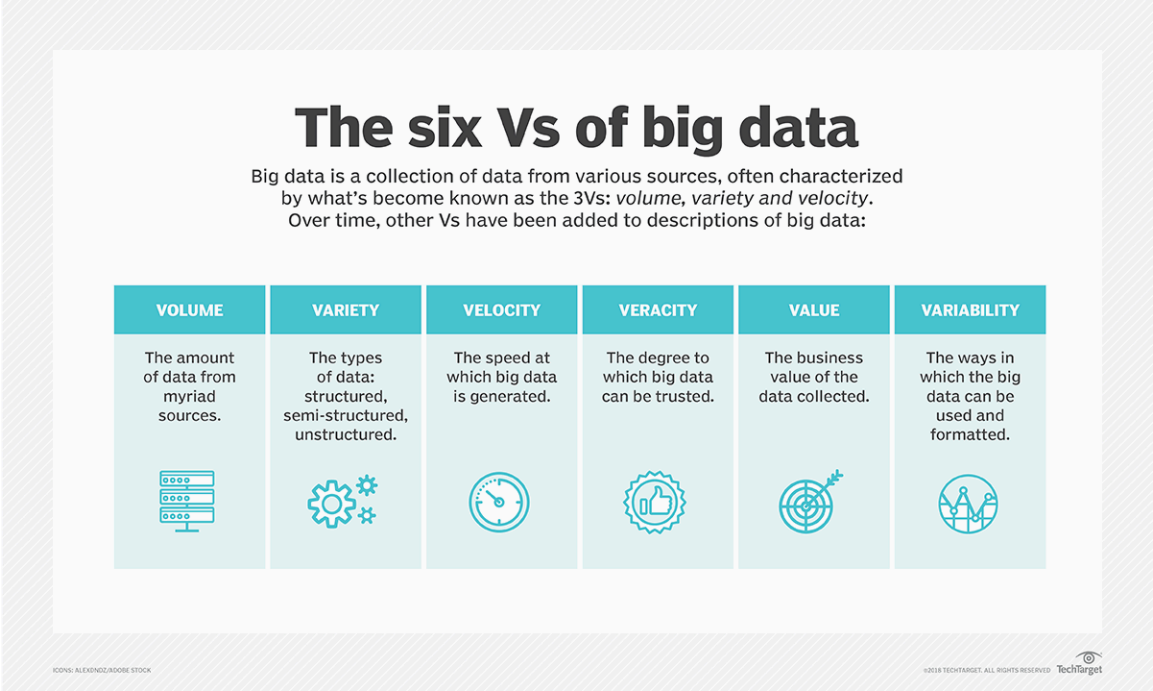
EL10: Biggish data

Big data



Biggish data

Our focus:

- Only tabular data
- Original data might fit into physical RAM but ...
- might be multiplied due to temporary necessity
- physical RAM might be constrained due to other processes/settings
- things might just take too much time ... and life is short ...

Software

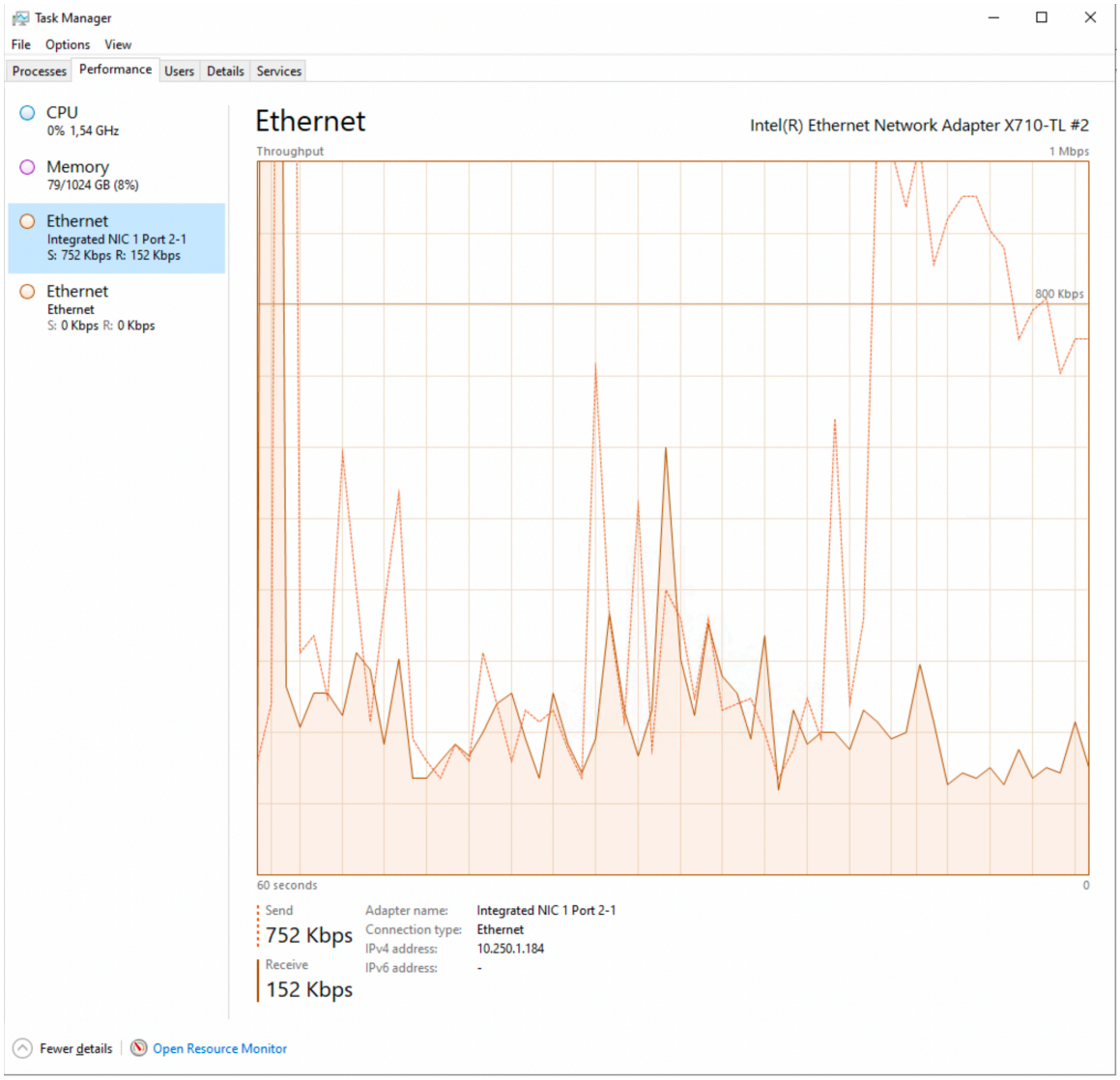
- SAS for big data files from register holders
- R for everything else
- Database?

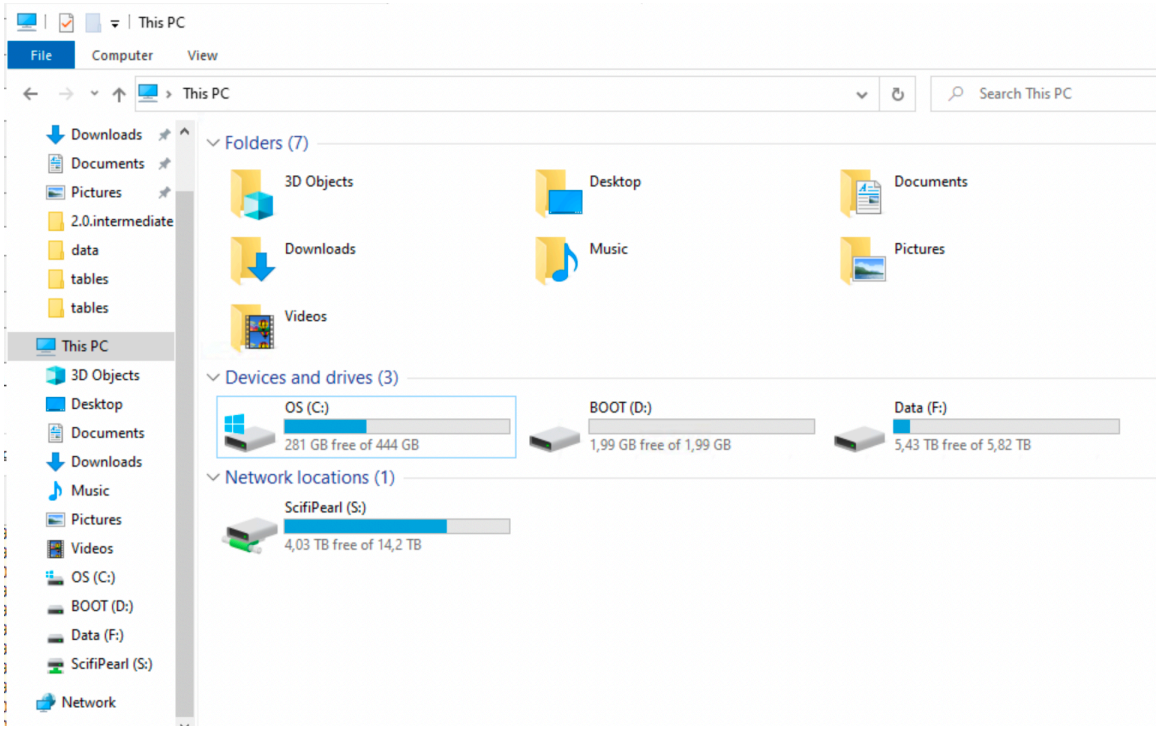
File storage

- Avoid reading big files over ethernet connection
  - data stored on network drive
- Usually faster to first copy file to local SSD drive

- `fs::file_copy(path, new_path)`
- (*"libuv provides a wide variety of cross-platform sync and async file system operations."*)
- obviously only if "local drive" is also in the (same) secure environment

- work with the project on disk
- move back
- Although not always possble with very large files it seems



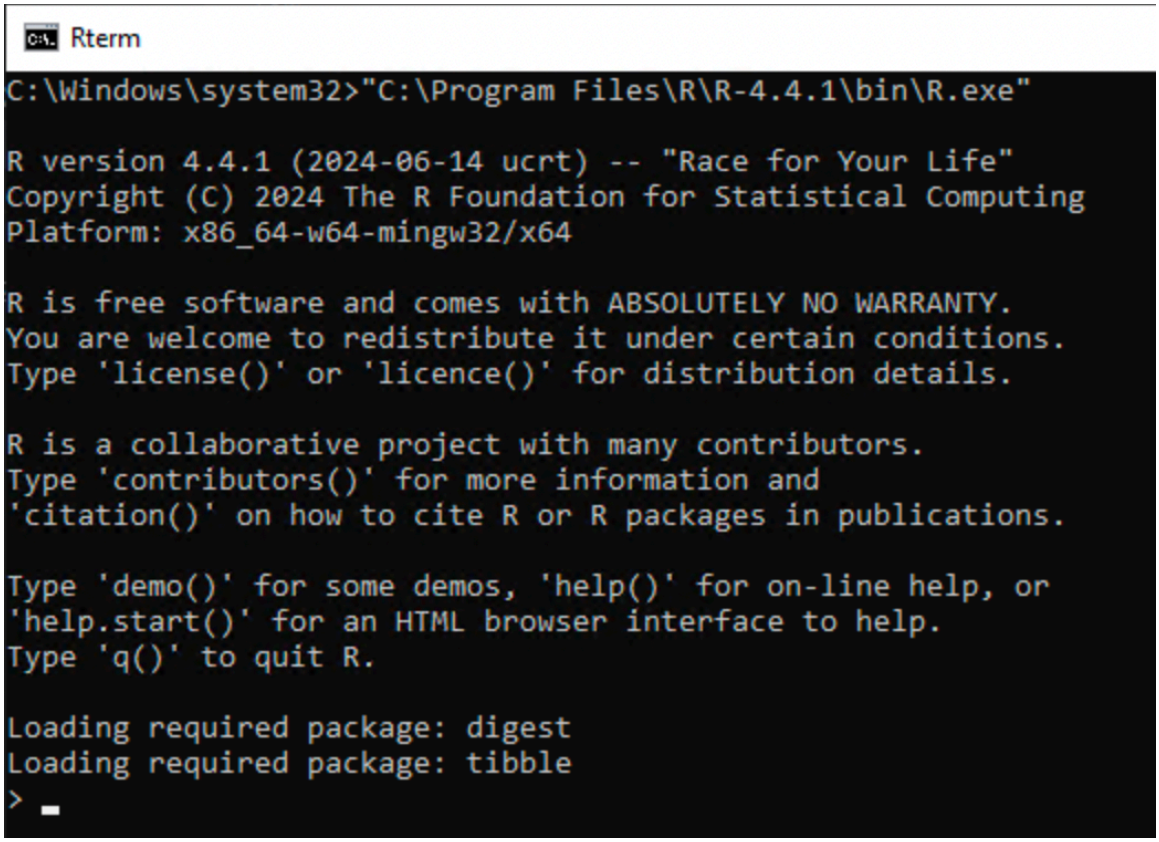

This PC
File
Computer
View
This PC
Search This PC
Downloads
Documents
Pictures
2.0.intermediate
data
tables
tables
This PC
3D Objects
Desktop
Documents
Downloads
Music
Pictures
Videos
OS (C:)
BOOT (D:)
Data (F:)
ScifiPearl (S:)
Network
Folders (7)
3D Objects
Desktop
Documents
Downloads
Music
Pictures
Videos
Devices and drives (3)
OS (C:)
281 GB free of 444 GB
BOOT (D:)
1,99 GB free of 1,99 GB
Data (F:)
5,43 TB free of 5,82 TB
Network locations (1)
ScifiPearl (S:)
4,03 TB free of 14,2 TB

Comparison vs base equivalents

fs functions smooth over some of the idiosyncrasies of file handling with base R functions:

- Vectorization. All **fs** functions are vectorized, accepting multiple paths as input. Base functions are inconsistently vectorized.
- Predictable return values that always convey a path. All **fs** functions return a character vector of paths, a named integer or a logical vector, where the names give the paths. Base return values are more varied: they are often logical or contain error codes which require downstream processing.
- Explicit failure. If **fs** operations fail, they throw an error. Base functions tend to generate a warning and a system dependent error code. This makes it easy to miss a failure.
- UTF-8 all the things. **fs** functions always convert input paths to UTF-8 and return results as UTF-8. This gives you path encoding consistency across OSes. Base functions rely on the native system encoding.
- Naming convention. **fs** functions use a consistent naming convention. Because base R's functions were gradually added over time there are a number of different conventions used (e.g. `path.expand()` vs `normalizePath()`; `Sys.chmod()` vs `file.access()`).

Still open?



ALTREP

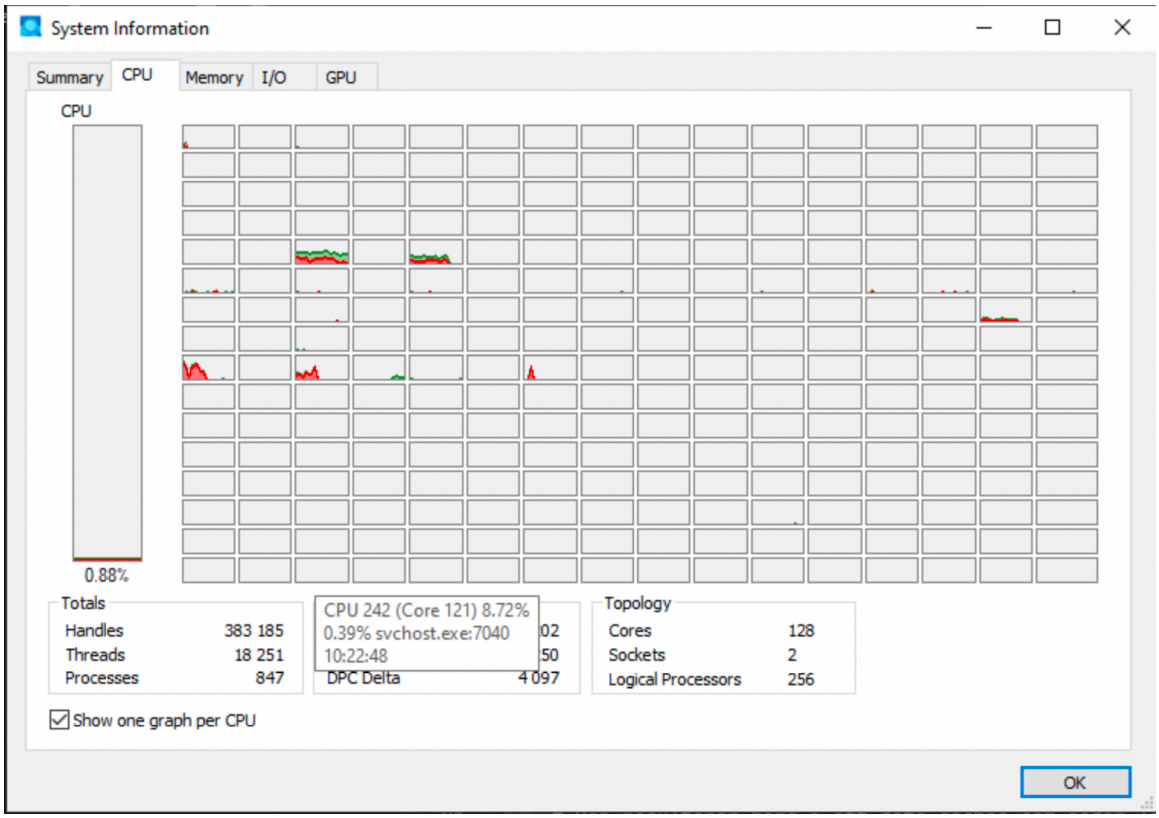
- **ALT**ernative **REP**resentation of R Objects
- Since R version 3.5.0
- Applies to the C-level representation of R objects under the hood
- Used by some base R packages/functions + specialized packages

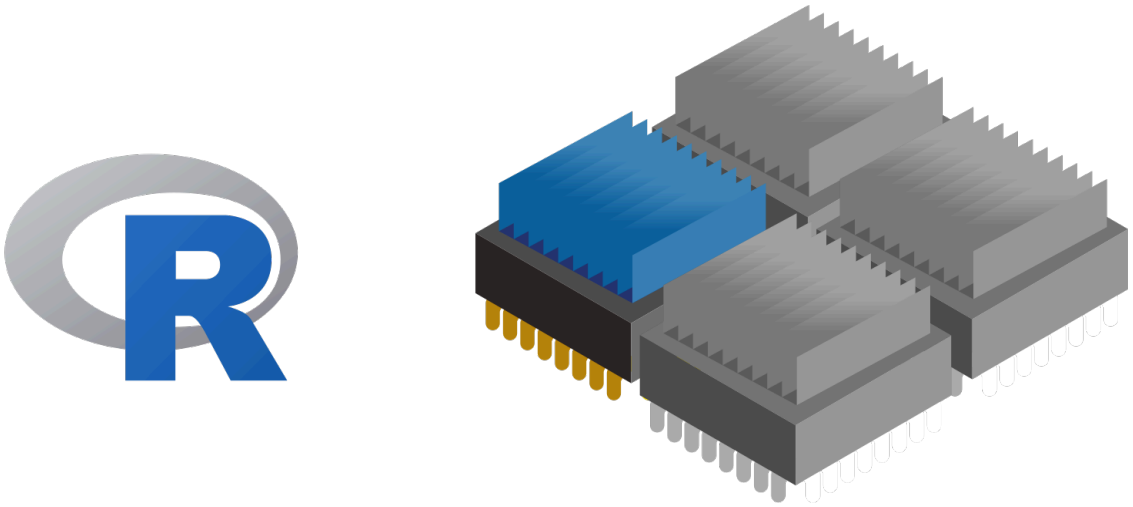
> allow vector data to be in a memory-mapped file; distributed, e.g. within Apache Spark or Hadoop; shared with other applications, e.g. with Apache Arrow; allow compact representation of arithmetic sequences; allow adding meta-data to objects; allow computations/ allocations to be deferred; support alternative representations of environments.

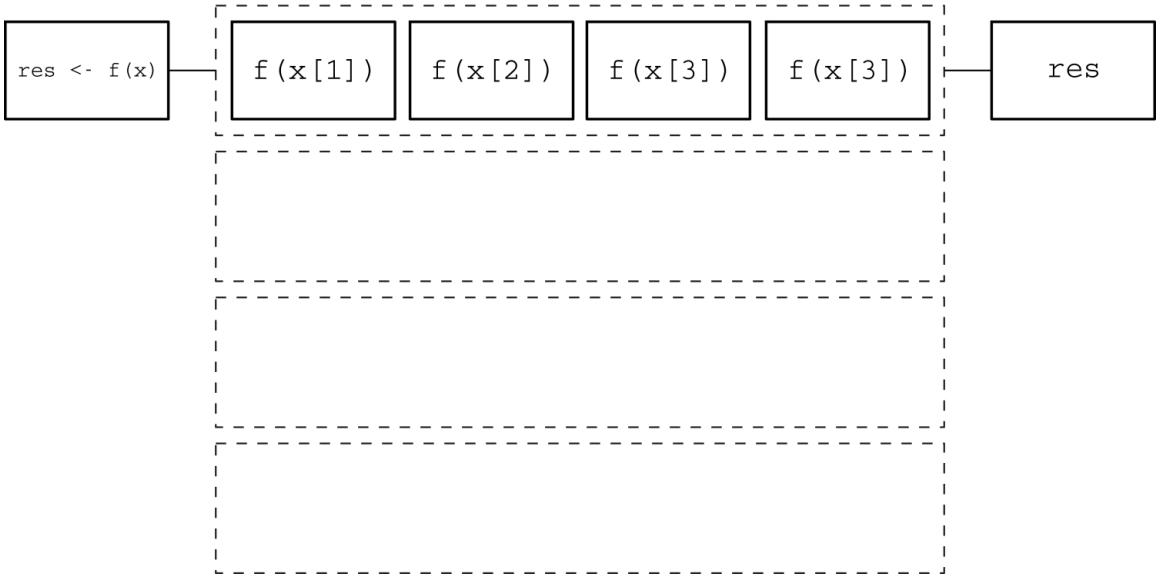
Dates

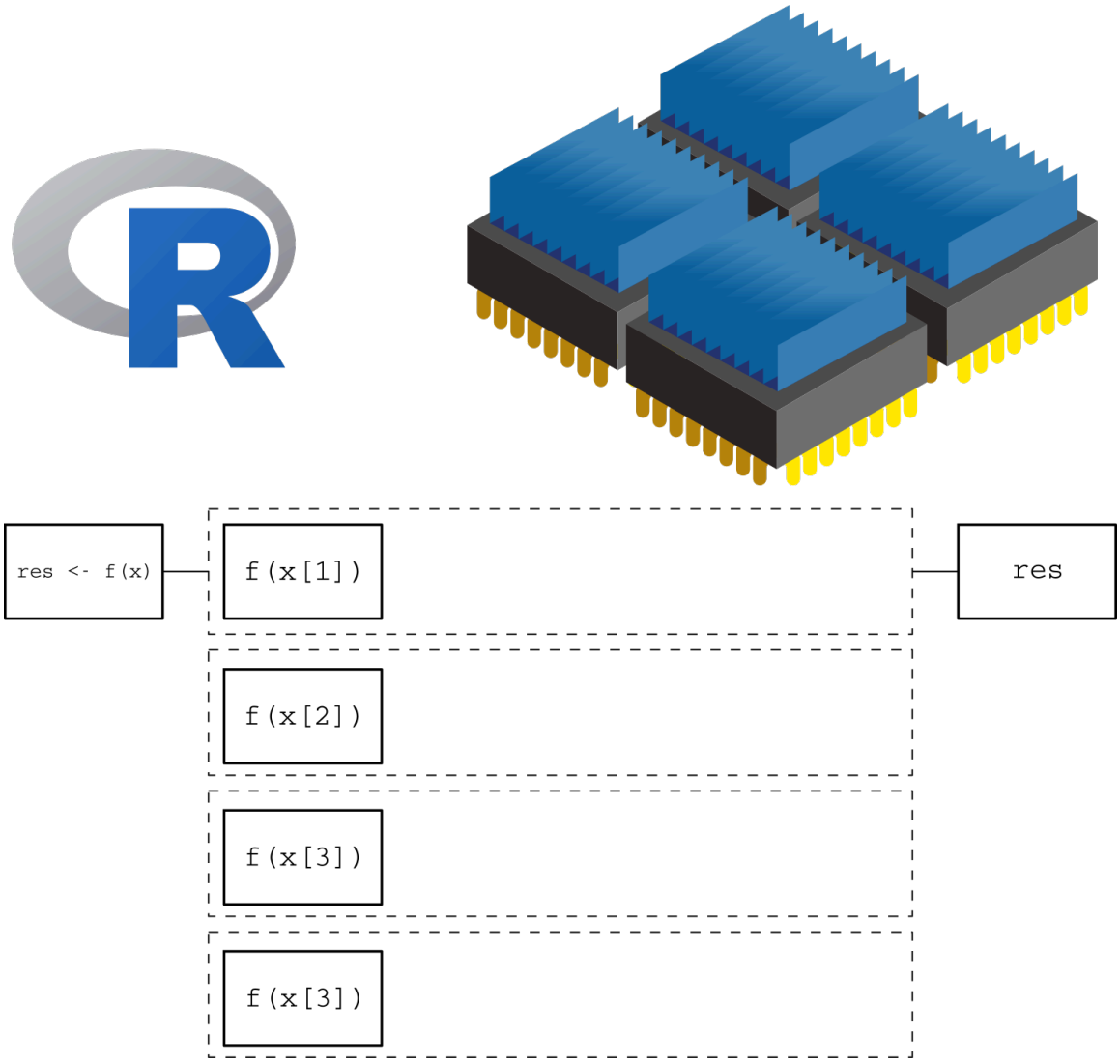
- Use `as.IDate()` instead of `as.Date()` if using `{data.table}`. Integer based.
- Use `{anytime}` (or maybe `{fasttime}`) if conversion is needed (`ALTREP`)

Paralellization






res <- f(x)
f(x[1])
f(x[2])
f(x[3])
f(x[3])
res



Paralellization

- R is single threaded by default
- Some packages and functions may use multithreading by default
- Some functions might have arguments such as `nthreads`, `ncores` etc
- To handle multicore/thread computations can be (very) difficult (I've heard)
- Easy if data can be divided and conquered (split-apply-combine)
- `{futureverse}` is my favorite (Henrik Bengtsson)
  - `{furrr}` is a a drop-in replacement for `{purrr}`

`{furrr}`

```
# How many cores do we have?
parallel::detectCores()
```

```
[1] 12
```

```
library(furrr)
```

```
Loading required package: future
```

```
# Save at least one for OS etc
plan(multisession, workers = parallel::detectCores() - 1)

1:10 |>
  future_map(rnorm, n = 10, .options = furrr_options(seed = 123)) |>
  future_map_dbl(mean)
```

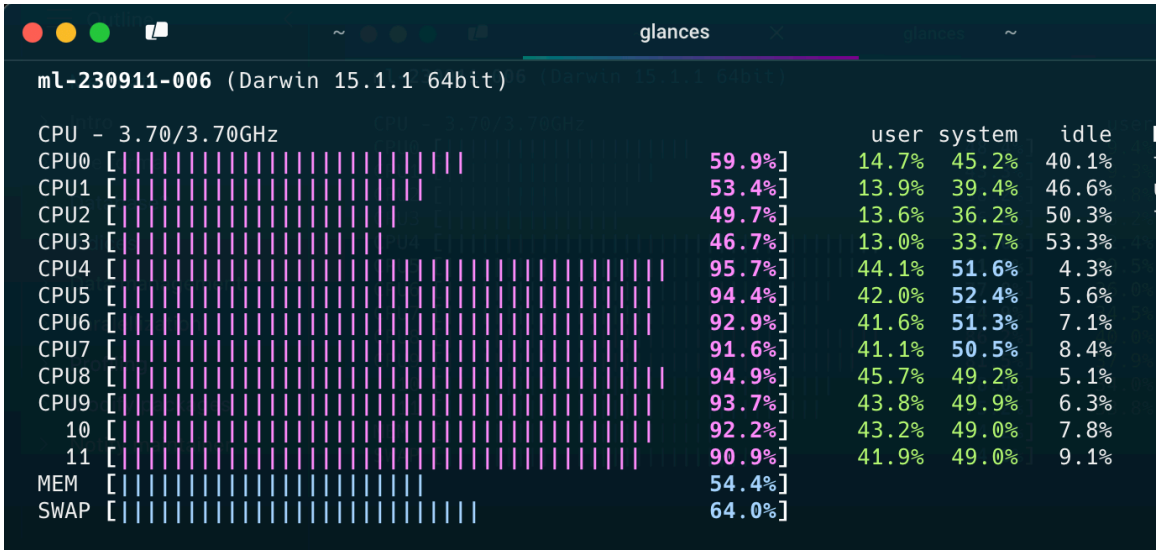
```
 [1] 1.180279 2.140442 2.909823 3.692207 5.058100 6.653926 7.065630 7.960713
 [9] 9.105674 9.766827
```

{mirai}

- Suposed to be up to 1,000 times "faster" compared to `{furrr}` (according to some metric ...)
- should not block the main process while executing (haven't tried)

> Designed for simplicity, a 'mirai' evaluates an R expression asynchronously in a parallel process, locally or distributed over the network, with the result automatically available upon completion.

```
library(mirai)
library(data.table)
daemons(11)

# data.table with 1 billion rows:
dt <- data.table(x = seq_len(1e9), y = rnorm(1e9))
# row sums (row wise )
mirai_map(dt, sum)[.flat]
```



{purrr}

- Introduced `in_paralell()` in 2025
- Based on mirai but without the specialised syntax
- Needs to be explicit on which objects/functions to export to each worker/daemon

```
library(purrr)
library(mirai)

# Set up parallel processing (6 background processes)
daemons(6)

# Sequential version
mtcars |> map_dbl(\(x) mean(x))
```

```
      mpg       cyl      disp        hp      drat        wt      qsec
20.090625  6.187500 230.721875 146.687500  3.596563  3.217250 17.848750
       vs        am      gear      carb
 0.437500  0.406250  3.687500  2.812500
```

```
#>    mpg    cyl   disp     hp   drat     wt   qsec     vs     am   gear
carb
#>  20.09   6.19 230.72 146.69   3.60   3.22  17.85   0.44   0.41   3.69
2.81

# Parallel version - just wrap your function with in_parallel()
mtcars |> map_dbl(in_parallel(\(x) mean(x)))
```

```
      mpg        cyl       disp         hp       drat         wt       qsec
 20.090625   6.187500 230.721875 146.687500   3.596563   3.217250  17.848750
       vs         am       gear       carb
  0.437500   0.406250   3.687500   2.812500
```

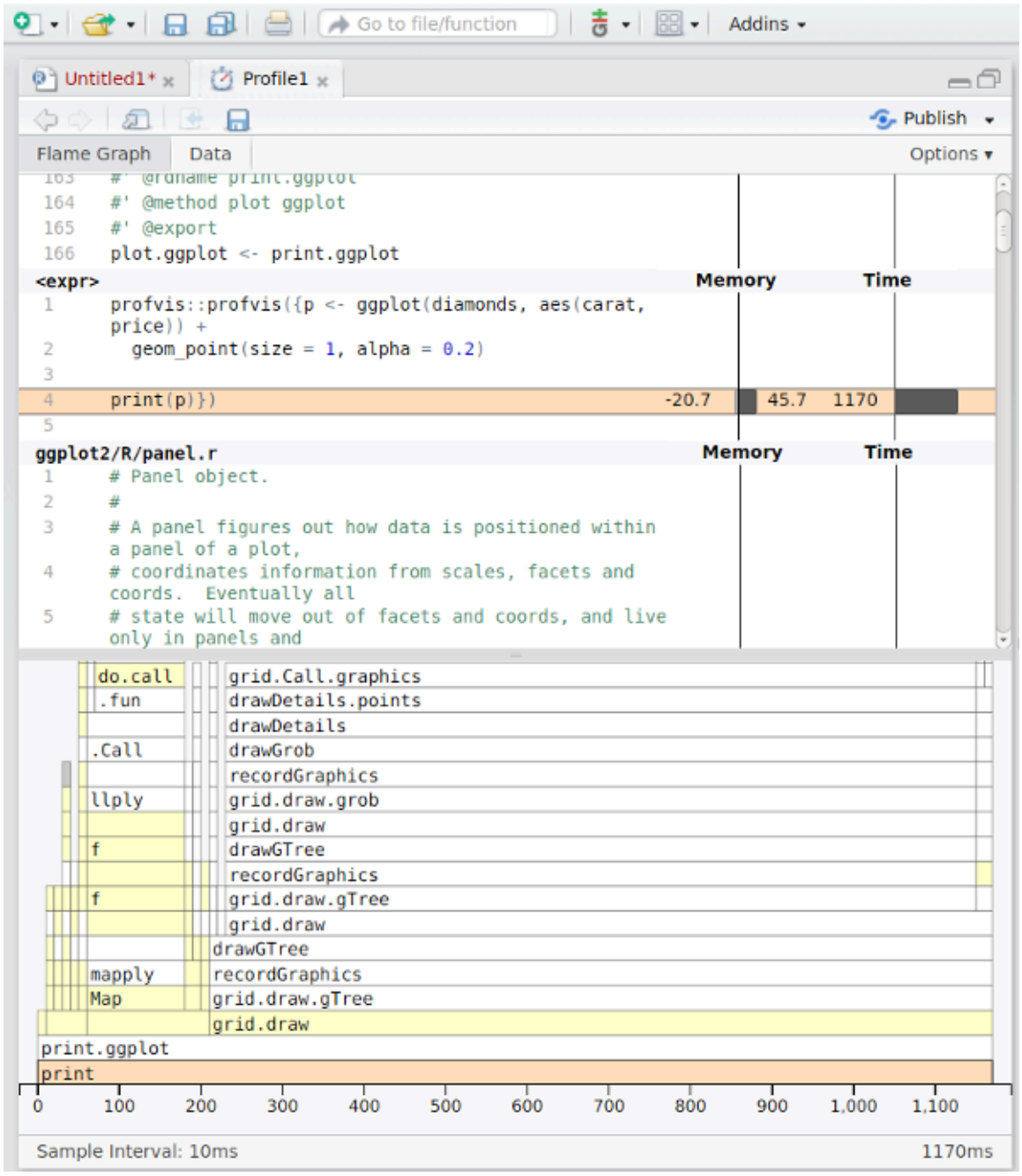
```
#>    mpg    cyl   disp     hp   drat     wt   qsec     vs     am   gear
carb
#>  20.09   6.19 230.72 146.69   3.60   3.22  17.85   0.44   0.41   3.69
2.81

# Don't forget to clean up when done
daemons(0)
```

Note

- Data must be copied for each paralell worker
- If all data is needed for each worker, it will be multiplied in memory
  - This might not be possible for big data
  - It also takes time
- Hence, there is no guarantee that the over all time will decrease
- Often a tradeof between no of workers/threads/deamons/cores and efficiency etc
- "Standard" computations in `{data.table}` is optimized for single core
- Message handling and progress bar etc is complicated

Profiling

- First version of R script might be inneficient
- Optimazation is difficult
- Might be suboptimal if made ad hoc
- `{profvis}` visualise time and memory usage for each step
- improve the most important step first
  - For example change `{base}` and `{tidyverse}` code to `{data.table}`
  - set keys
  - Efficient handling of dates and strings
- itterate
- Integrated in RStudio (but not yet Positron)



Modify packages

Modify packages

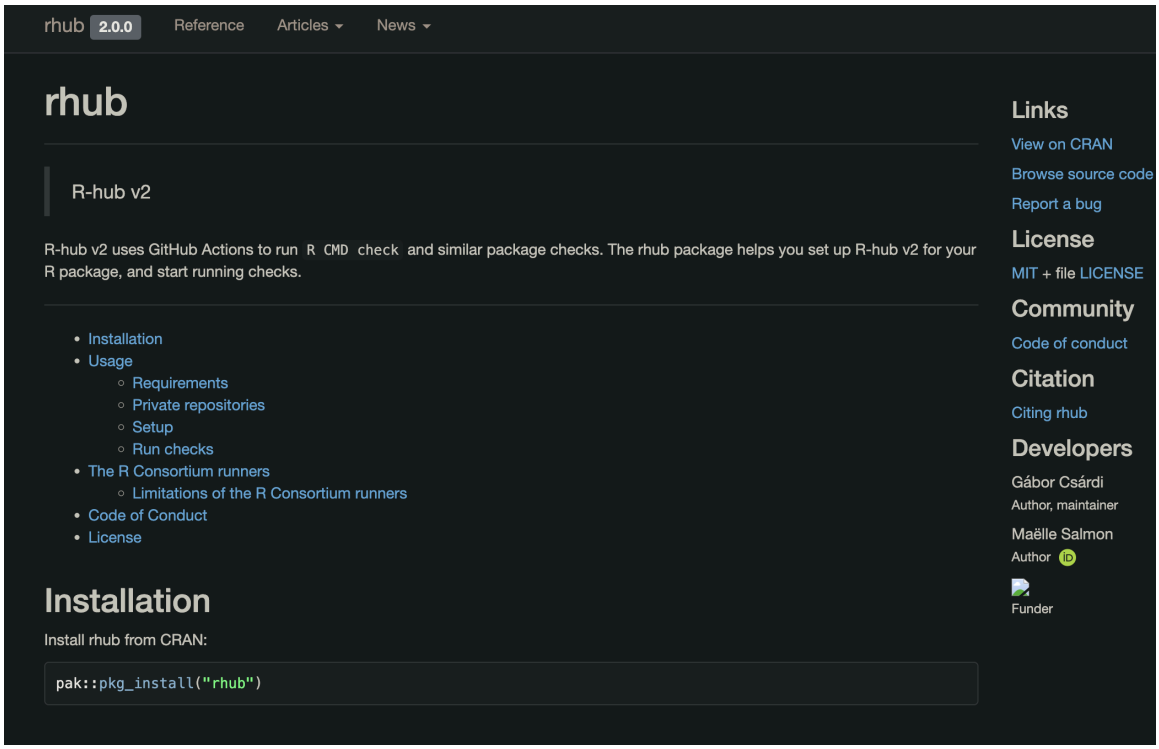
- Functions from packages might be inefficient
- Profile those as well
- Clone package from GitHub if available or download source code from CRAN
- Improve

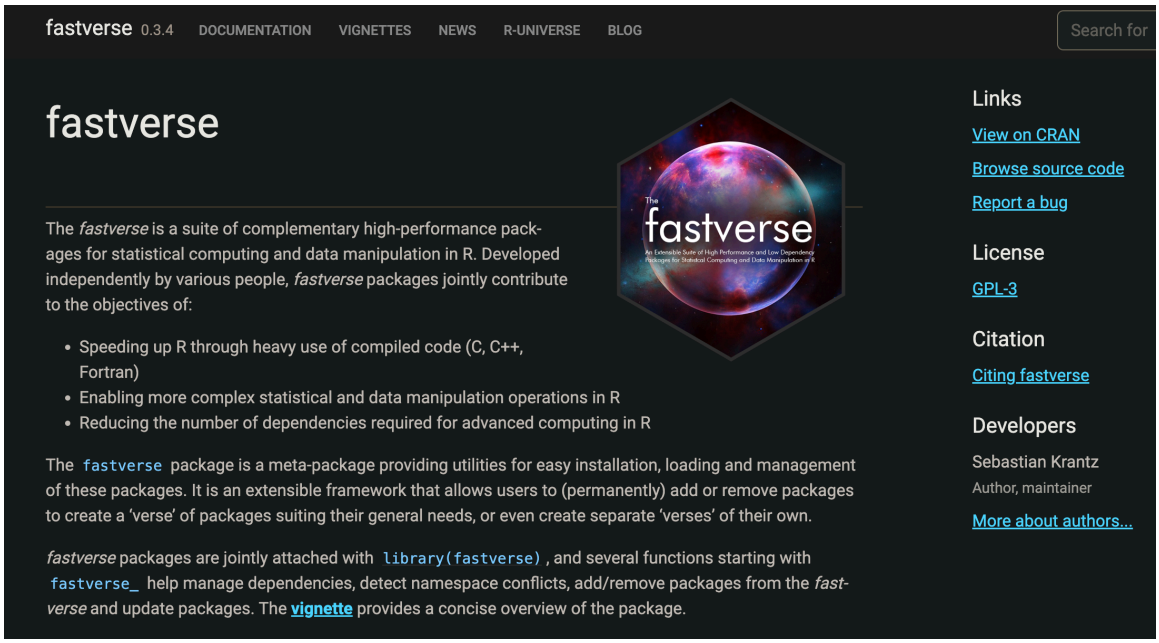
- Just load the modified function in global environment
  - might need to modify internal calls to `:::`-accesed functions
- Or rebuild/install the package if more convenient

> ! Notify maintainer
>
> If you find ways to improve a package, the maintainer might be eager to hear! Suggest pull request or open GitHub issue!

C-code changes

- If the package use C-code it is probably already very fast
- If not, and you can fix it, the package must be re-compiled
- OS dependent
- Might be tricky if using restricted environments (TRE?)
- Can still be done by rhub (might need to change maintainer-e-mail temporarily)





Bibliography